Machine Learning Lab Programs Solutions:

1. Write a python program to compute Central Tendency Measures: Mean, Median, Mode Measure of Dispersion: Variance, Standard Deviation

Solution :

Understanding Basic Statistics

Statistical analysis often involves understanding the central tendency and the dispersion of data. Below are the definitions, along with the formulas used for these measures.

1. Central Tendency Measures

- **Mean (μ):** The arithmetic average of a set of values. It is calculated by summing all the data points and dividing by the number of data points.

$$\text{Mean}(\mu) = \frac{\sum_{i=1}^{n} x_i}{n}$$

- **Median:** The median is the middle value in a sorted list of numbers. If the number of observations is even, the median is the average of the two middle numbers.

$$\text{Median} = \begin{cases} \text{Middle value}, & \text{if } n \text{ is odd} \\ \frac{x_{n/2} + x_{(n/2)+1}}{2}, & \text{if } n \text{ is even} \end{cases}$$

where $x_{n/2}$ and $x_{(n/2)+1}$ are the middle elements after sorting the data.

- **Mode:** The mode is the value that appears most frequently in a dataset. Mode = most frequent value in the dataset

2. Measures of Dispersion

- **Variance (σ^2):** Variance measures the spread of the data points around the mean. It is calculated as the average of the squared differences between each data point and the mean.

$$\text{Variance}(\sigma^2) = \frac{\sum_{i=1}^{n}(x_i - \mu)^2}{n-1}$$

where x_i are the individual data points, μ is the mean of the data, and n is the total number of data points.

- **Standard Deviation (σ):** The standard deviation is the square root of the variance, providing a measure of the average distance from the mean. It is in the same units as the data, making it more interpretable compared to variance.

$$\text{Standard Deviation}(\sigma) = \sqrt{\frac{\sum_{i=1}^{n}(x_i - \mu)^2}{n-1}}$$

These measures provide insights into the distribution, central point, and spread of data, helping to understand the data's overall behaviour.

Source Code:

File name: Stat_Measures.py

```
# Prompt user to enter data
user_input = input("Enter numbers separated by commas: ")

# Convert the input string to a list of integers
numbers = [int(num) for num in user_input.split(',')]
```

```
# Calculate the number of elements
count = len(numbers)

# Calculate the mean (average)
sum_of_numbers = sum(numbers)
mean_value = sum_of_numbers / count

# Sort the list for median calculation
sorted_numbers = sorted(numbers)

# Calculate the median
if count % 2 == 0:
    median_value = (sorted_numbers[count // 2 - 1] + sorted_numbers[count // 2]) / 2
else:
    median_value = sorted_numbers[count // 2]

# Calculate the mode
frequency_dict = {}
highest_frequency = 0

for num in numbers:
    if num in frequency_dict:
        frequency_dict[num] += 1
    else:
        frequency_dict[num] = 1
    if frequency_dict[num] > highest_frequency:
        highest_frequency = frequency_dict[num]

# Get all numbers with the highest frequency
modes = [num for num, freq in frequency_dict.items() if freq == highest_frequency]

# Calculate variance
squared_diff_sum = sum((num - mean_value) ** 2 for num in numbers)
variance_value = squared_diff_sum / count

# Calculate standard deviation
std_deviation = variance_value ** 0.5

# Display results
print(f"Mean = {mean_value}")
print(f"Median = {median_value}")
print(f"Mode = {modes if len(modes) < count else 'No mode (all values are unique)'}")
print(f"Variance = {variance_value}")
print(f"Standard Deviation = {std_deviation}")
```

Output:

Sample Run1:

$ python3 Stat_Measures.py

Enter numbers separated by commas: 1,2,3,4,4,5,6
Mean = 3.5714285714285716
Median = 4
Mode = [4]
Variance = 2.5306122448979593
Standard Deviation = 1.5907898179514348

Sample Run2:

$ python3 Stat_Measures.py
Enter numbers separated by commas: 10,20,30,30,40,40
Mean = 28.333333333333332
Median = 30.0
Mode = [30, 40]
Variance = 113.88888888888887
Standard Deviation = 10.671873729054747

2. Study of Python Basic Libraries such as Statistics, Math, Numpy and Scipy

Python is a powerful programming language widely used for data analysis, scientific research, and statistical computing. Several libraries in Python make it particularly strong in these areas:

1. Statistics Library:

The statistics module provides functions to perform basic statistical operations like calculating the mean, median, mode, variance, and standard deviation.

Key Functions:

- **mean(data):** Returns the arithmetic mean of the data.
- **median(data):** Returns the median value.
- **mode(data):** Returns the mode (most frequent value).
- **variance(data):** Returns the variance of the data.
- **stdev(data):** Returns the standard deviation.

Source code:

File Name: Stat_Lib.py

```
import statistics
# Input data from user
data=input("Enter data separated by comma:")
data=[int(x) for x in data.split(',')]
# Compute Mean
mean=statistics.mean(data)
# Compute Median
median=statistics.median(data)
# Compute Mode
mode=statistics.mode(data)
# Compute Standard Deviation
std_dev=statistics.stdev(data)
# Compute Variance
variance=statistics.variance(data)
# Output results
print(f"Mean={mean}")
print(f"Median={median}")
print(f"Mode={mode}")
print(f"Standard Deviation={std_dev}")
print(f"Variance={variance}")
```

Output:

Sample Run:

$python3 Stat_Lib.py.py

Enter data separated by comma:1,2,3,4,4,5,6
Mean=3.5714285714285716
Median=4
Mode=4
Standard Deviation=1.7182493859684491
Variance=2.9523809523809526

---

2. Math Library:
The math module offers a range of mathematical functions that operate on numbers and perform tasks such as trigonometric calculations, logarithms, and square roots.
Key Functions:

- **math.ceil(x):** Returns the smallest integer greater than or equal to x.
- **math.factorial(x):** Returns the factorial of a non-negative integer x.
- **math.exp(x):** Returns exe^xex, where eee is the base of natural logarithms (approximately 2.718).
- **math.sqrt(x):** Returns the square root of x.
- **math.floor(x):** Returns the largest integer less than or equal to x.
- **math.gcd(x, y):** Returns the greatest common divisor (GCD) of two integers x and y.
- **math.pow(x, y):** Returns xyx^yxy (x raised to the power of y).
- **math.radians(angle):** Converts an angle from degrees to radians.
- **math.sin(x):** Returns the sine of x, where x is in radians.
- **math.cos(x):** Returns the cosine of x, where x is in radians.
- **math.tan(x):** Returns the tangent of x, where x is in radians.

☛ **Library Installation:**
Don't need to install **math module** library, because this is a built-in module in Python.

Source Code:
File Name: Math_Lib.py

```
import math
x=int(input("Enter a number:"))
result1=math.ceil(x)
result2=math.factorial(x)
result3=math.exp(x)
result4=math.sqrt(x)
result5=math.floor(x)
n1=int(input("Enter number1:"))
n2=int(input("Enter number2:"))
result6=math.gcd(n1,n2)
result7=math.pow(n1,n2)
angle=int(input("Enter an angle:"))
a=math.radians(angle)
result8=math.sin(a)
result9=math.cos(a)
result10=math.tan(a)
print(f"Using math.ceil({x}): {result1}")
print(f"Using math.factorial({x}):{result2}")
print(f"Using math.exp({x}):{result3}")
print(f"Using math.sqrt({x}):{result4}")
print(f"Using math.floor({x}):{result5}")
print(f"Using math.gcd({n1},{n2}):{result6}")
print(f"Using math.pow({n1},{n2}):{result7}")
print(f"Using math.sin({angle}):{result8}")
print(f"Using math.cos({angle}):{result9}")
print(f"Using math.tan({angle}):{result10:.15f}")
```

Output:

Sample Run:

```
$python3 Math_Lib.py.py

Enter a number:4
Enter number1:3
Enter number2:2
Enter an angle:1
Using math.ceil(4): 4
Using math.factorial(4):24
Using math.exp(4):54.598150033144236
Using math.sqrt(4):2.0
Using math.floor(4):4
Using math.gcd(3,2):1
Using math.pow(3,2):9.0
Using math.sin(1):0.01745240643728351
Using math.cos(1):0.9998476951563913
Using math.tan(1):0.017455064928218
```

---

3. Numpy Library:

numpy is a library for numerical computing in Python. It is used for working with arrays and matrices, along with a collection of mathematical functions to operate on these data structures.

Key Functions:

- np.array(data): Creates a numpy array from the given list of elements.
- reshape(rows, cols): Reshapes an array into the specified number of rows and columns.
- np.ones((rows, cols)): Creates an array filled with ones of the specified shape (number of rows and columns).
- np.zeros((rows, cols)): Creates an array filled with zeros of the specified shape (number of rows and columns).
- np.transpose(array): Returns the transpose of the given array, flipping rows into columns and vice versa.
- np.min(array): Finds the minimum value in the array.
- np.max(array): Finds the maximum value in the array.

Library Installation:

To install Numpy library, Open **Command Prompt** or **Terminal** and execute the following commands

```
# To install numpy on Linux
$ sudo apt install python3-numpy
(or)
$ pip3 install numpy

# To install numpy on Windows
pip install numpy
```

Source Code:

File Name: Numpy_Lib.py

```
import numpy as np
# Get number of rows and columns from user
rows=int(input("Enter number of rows:"))
cols=int(input("Enter number of columns:"))
# Get array elements from user
elements=list(map(float, input(f"Enter {rows*cols} elements separated by spaces:").split()))
# Create an array from user input
user_array=np.array(elements).reshape(rows,cols)
# Create an array of ones and zeros
ones_array=np.ones((rows, cols))
zeros_array=np.zeros((rows, cols))
# Transpose the user array
transposed_array=np.transpose(user_array)
```

```
# Reshape the user array (to a flat array and back to the original shape for demonstration)
reshaped_array = user_array.reshape(rows * cols)
min_value=np.min(elements)
max_value=np.max(elements)
# Display the results
print("User Array:\n",user_array)
print("Ones Array:\n",ones_array)
print("Zeros Array:\n",zeros_array)
print("Transposed Array:\n",transposed_array)
print("Reshaped Array (flattened):\n",reshaped_array)
print("Minimum Value:",min_value)
print("Maximum Value:",max_value)
```

Output:

```
Sample Run:
-------------
$python3 Numpy_Lib.py
Enter number of rows:2
Enter number of columns:2
Enter 4 elements separated by spaces:1 2 3 4
User Array:
 [[1. 2.]
 [3. 4.]]
Ones Array:
 [[1. 1.]
 [1. 1.]]
Zeros Array:
 [[0. 0.]
 [0. 0.]]
Transposed Array:
 [[1. 3.]
 [2. 4.]]
Reshaped Array (flattened):
 [1. 2. 3. 4.]
Minimum Value: 1.0
Maximum Value: 4.0
```

---

4. Scipy Library:

Scipy builds on numpy and provides a wide range of advanced mathematical, scientific, and engineering functions, including optimization, integration, interpolation, and statistical operations.

Key Functions:

From scipy.constants:

- **constants.minute:** Returns the number of seconds in a minute (60.0).
- **constants.hour:** Returns the number of seconds in an hour (3600.0).
- **constants.inch:** Returns the length of an inch in meters (0.0254).
- **constants.liter:** Returns the volume of a liter in cubic meters (0.001).

From scipy.special:

- **special.exp10(x):** Calculates 10x, where x is the exponent provided by the user.
- **special.sindg(angle):** Returns the sine of an angle (in degrees).
- **special.cosdg(angle):** Returns the cosine of an angle (in degrees).

From scipy.linalg:

- **linalg.det(matrix):** Calculates the determinant of a square matrix. If the input matrix is not square, the determinant cannot be calculated.

From numpy:

- **np.array(data):** Creates a NumPy array from the provided list of elements.
- **reshape(rows, cols):** Reshapes the array into the specified number of rows and columns.

Library Installation:

To install scipy library, Open **Command Prompt** or **Terminal** and execute the following commands

```
# To install scipy on Linux
$ sudo apt install python3-scipy
(or)
$ pip3 install scipy

# To install scipy on Windows
pip install scipy
```

Source Code:
File Name: Scipy_Lib.py

```
from scipy import linalg
import numpy as np
from scipy import special
from scipy import constants
# Print constants
print("Seconds in a minute:",constants.minute) # 60.0
print("Seconds in an hour:",constants.hour) # 3600.0
print("Length of an inch in meters:",constants.inch) # 0.0254
print("Volume of a liter in cubic meters:",constants.liter) # 0.001
# Dynamic input for exponentiation
exp_input = float(input("Enter the exponent for 10^x:"))
print(f"10^{exp_input} =",special.exp10(exp_input))
# Dynamic input for angles in degrees
angle=float(input("Enter an angle for sine and cosine(in degrees):"))
#Calculate sine and cosine of the input angles
print(f"sin({angle} degrees):",special.sindg(angle))
print(f"cos({angle} degrees):",special.cosdg(angle))
# Dynamic input for matrix
rows = int(input("Enter the number of rows for the matrix:"))
cols = int(input("Enter the number of columns for the matrix:"))
mat = list(map(float, input(f"Enter {rows * cols} elements separated by spaces:").split()))
# Convert the flat list into a 2D NumPy array
mat=np.array(mat).reshape((rows, cols))
# Calculate the determinant if the matrix is square
if rows == cols:
        x = linalg.det(mat)
        print(f'Determinant of the matrix\n{mat}\n is: {x}')
else:
        print("Determinant can only be calculated for square matrices.")
```

Output:

```
Sample Run:
----------
$python Scipy_Lib.py
Seconds in a minute: 60.0
Seconds in an hour: 3600.0
Length of an inch in meters: 0.0254
Volume of a liter in cubic meters: 0.001
Enter the exponent for 10^x:2
10^2.0 = 100.0
Enter an angle for sine and cosine(in degrees):90
sin(90.0 degrees): 1.0
cos(90.0 degrees): -0.0
Enter the number of rows for the matrix:2
Enter the number of columns for the matrix:2
Enter 4 elements separated by spaces:2 3 4 5
```

Determinant of the matrix
[[2. 3.]
 [4. 5.]]
 is: -2.0000000000000004

3. Study of Python Libraries for ML application such as Pandas and Matplotlib

Solution:

1. Pandas

Pandas is a fast, powerful, and flexible library used for data analysis and manipulation. It provides data structures like DataFrames and Series that are widely used in ML for managing structured data.

Key Features of Pandas

1. **Data Handling:** Create, modify, and process structured data efficiently.
2. **File Input/Output:** Read from and write to formats like CSV, Excel, JSON,SQL, etc.
3. **Data Cleaning:** Handle missing data, filter rows/columns, and apply transformations.
4. **Data Aggregation:** Group data for summarization and analysis.

Key Functions:

1. **pandas.read_csv(file_path)**

a. Reads a CSV file into a Pandas DataFrame. b. Parameters: i. file_path: The path of the CSV file. c. Returns: A Pandas DataFrame.

2. **df.loc[[row_indices]]**

a. Accesses specified rows (and optionally columns) in the DataFrame using labels or indices. b. Parameters:
i. row_indices: A list of row indices to fetch.

3. **df.head(n)**

a. Displays the first n rows of the DataFrame (default is 5).

4. **df.tail(n)**

a. Displays the last n rows of the DataFrame (default is 5).

5. **df.isnull()**

a. Returns a DataFrame of the same shape, indicating the presence of missing values.

6. **df.info()**

a. Prints a summary of the DataFrame, including data types, non-null values, and memory usage.

Library Installation:

To install Pandas library, Open **Command Prompt** or **Terminal** and execute the following commands

```
# To install Pandas on Linux
$ sudo apt install python3-pip
$ pip3 install pandas

# To install Pandas on Windows
pip install pandas
```

CSV file : "students.csv"

```
Student_NO,Name,Branch,Year,Contact_NO
1201,Raghu,IT,III,1234
1202,Sai,IT,III,1234
1203,Sravan,IT,III,1234
1204,Karthik,IT,III,1234
1205,Gaanesh,IT,III,1234
1206,Vamshi,IT,III,1234
1207,Shiva,IT,III,1234
1208,Racha,IT,III,1234
1209,Abu,IT,III,1234
1210,Nithish,IT,III,1234
```

Source Code:

File Name: Pandas_Lib.py

```
import pandas
df=pandas.read_csv('students.csv')
print(df)
print(df.loc[[0,1]])
print(df.head())
print(df.tail())
print(df.isnull())
print(df.info())
```

Output:

Sample Run:

```
--------------
$python3 Pandas_Lib.py
   Student_NO     Name Branch  Year  Contact_NO
0        1201    Raghu     IT  III        1234
1        1202      Sai     IT  III        1234
2        1203   Sravan     IT  III        1234
3        1204  Karthik     IT  III        1234
4        1205  Gaanesh     IT  III        1234
5        1206   Vamshi     IT  III        1234
6        1207    Shiva     IT  III        1234
7        1208    Racha     IT  III        1234
8        1209      Abu     IT  III        1234
9        1210  Nithish     IT  III        1234
   Student_NO   Name Branch Year  Contact_NO
0        1201  Raghu     IT  III        1234
1        1202    Sai     IT  III        1234
   Student_NO     Name Branch Year  Contact_NO
0        1201    Raghu     IT  III        1234
1        1202      Sai     IT  III        1234
2        1203   Sravan     IT  III        1234
3        1204  Karthik     IT  III        1234
4        1205  Gaanesh     IT  III        1234
   Student_NO     Name Branch Year  Contact_NO
5        1206   Vamshi     IT  III        1234
6        1207    Shiva     IT  III        1234
7        1208    Racha     IT  III        1234
8        1209      Abu     IT  III        1234
9        1210  Nithish     IT  III        1234
   Student_NO   Name  Branch   Year  Contact_NO
0       False  False   False  False       False
1       False  False   False  False       False
2       False  False   False  False       False
3       False  False   False  False       False
4       False  False   False  False       False
5       False  False   False  False       False
6       False  False   False  False       False
7       False  False   False  False       False
8       False  False   False  False       False
9       False  False   False  False       False

RangeIndex: 10 entries, 0 to 9
Data columns (total 5 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   Student_NO  10 non-null     int64
 1   Name        10 non-null     object
```

```
 2   Branch      10 non-null     object
 3   Year        10 non-null     object
 4   Contact_NO  10 non-null     int64
dtypes: int64(2), object(3)
memory usage: 532.0+ bytes
None
```

4. Write a Python program to implement Simple Linear Regression

Solution :

Linear regression is a statistical method used to model the relationship between a dependent variable and one or more independent variables. It provides valuable insights for prediction and data analysis. This article will explore its types, assumptions, implementation, advantages, and evaluation metrics.

Simple Linear Regression:

Simple linear regression is the simplest form of linear regression and it involves only one independent variable and one dependent variable.

The equation for simple linear regression is:

$y = \beta 0 + \beta 1 X$

where:

y is the dependent variable

X is the independent variable

β0 is the intercept

β1 is the slope

Step-1: Data Pre-processing

The first step for creating the Simple Linear Regression model is data pre-processing

Step-2: Fitting the Simple Linear Regression to the Training Set:

Now the second step is to fit our model to the training dataset. To do so, we will import the LinearRegression class of the linear_model library from the scikit learn.

Step: 3. Prediction of test set result:

dependent (salary) and an independent variable (Experience). So, now, our model is ready to predict the output for the new observations. In this step, we will provide the test dataset (new observations) to the model to check whether it can predict the correct output or not.

Step: 4. visualizing the results:

Now in this step, we will visualize the training set result. To do so, we will use the scatter() function of the pyplot library, which we have already imported in the pre-processing step. The scatter () function will create a scatter plot of observations.

Library Installation:

To install required library files, Open **Command Prompt** or **Terminal** and execute the following commands

```
$ pip install scikit-learn

$ pip install pandas

$ pip install matplotlib
```

CSV file : "salary_data.csv"

```
age,salary
22,30000
25,35000
30,45000
35,50000
40,60000
45,65000
50,70000
55,80000
```

Source Code:

File Name: Linear_Regression.py

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error,mean_squared_error,r2_score

#Load the data from the CSV file
data = pd.read_csv('salary_data.csv')
x= data[['age']] # Independent variable
y = data['salary'] # Dependent variable

#Split the data into training and testing sets
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2,random_state=1)

model=LinearRegression()
model.fit(x_train, y_train)

y_pred=model.predict(x_test)

#Model coefficients
print(f"Intercept (a0): {model.intercept_}")
print(f"Slope (a1): {model.coef_}")
r2=r2_score(y_test, y_pred)

#Print evaluation metrics
print (f"R-squared Score: ",r2)

#User input for age
user_age = float(input("Enter age to predict salary: "))

#Predict salary for the given age
predicted_salary = model.predict(pd.DataFrame([[user_age]], columns=['age']))
```

```
print (f"The predicted salary for age {user_age} is:",predicted_salary)
plt.scatter (x_test, y_test, color='blue')
plt.plot(x_test, y_pred, color='red', linewidth=2, label='Predicted line')
plt.scatter (user_age, predicted_salary, color='green', s=100, label='User Prediction')
plt.xlabel('Age')
plt.ylabel('Salary')
plt.title('Simple Linear Regression: Age vs Salary')
plt.legend()
plt.show()
```

Output:

Sample Run:

$ python3 Linear_Regression.py

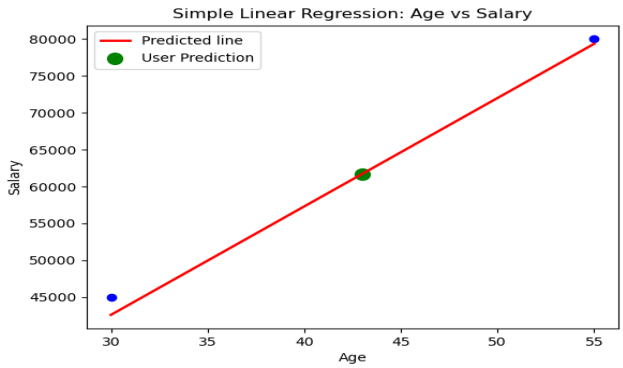
Intercept (a0): -1522.5102319235884

Slope (a1): [1470.66848568]

R-squared Score: 0.9899167950543547

Enter age to predict salary: 43

The predicted salary for age 43.0 is: [61716.23465211]



---

5. Implementation of Multiple Linear Regression for House Price Prediction using sklearn

Solution :

Multiple Linear Regression attempts to model the relationship between two or more features and a response by fitting a linear equation to observed data. The steps to perform multiple linear Regression are almost similar to that of simple linear Regression. The Difference Lies in the evaluation. We can use it to find out which factor has the highest impact on the predicted output and how different variables relate to each other.

Y = b0 + b1 * x1 + b2 * x2 + b3 * x3 + …… bn * xn

Y = Dependent variable

x1, x2, x3, …… xn = multiple independent variables

Step-1: Data Pre-processing
Step-2: Fitting our MLR model to the Training set
Step-3: Prediction of test set result
Step-4: visualizing the results

Library Installation:
To install required library files, Open **Command Prompt** or **Terminal** and execute the following commands

```
$ pip install scikit-learn

$ pip install pandas

$ pip install numpy

$ pip install matplotlib
```

CSV file : "house_data.csv"

```
price,area,bedrooms,floors,age
4500000,1500,3,2,5
7000000,2500,4,3,4
3000000,1200,2,1,2
5500000,1800,3,2,4
9000000,3000,5,4,5
3500000,1400,3,2,5
6000000,2200,4,3,1
10000000,3500,5,4,4
4000000,1600,3,2,3
6500000,2000,4,3,4
```

Source Code:
File Name: Multiple_Linear_Regression.py

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
from matplotlib.ticker import FuncFormatter

data = pd.read_csv('house_data.csv')
X = data.drop('price', axis=1)
y = data['price']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,random_state=42)
model = LinearRegression().fit(X_train, y_train)
y_pred = model.predict(X_test)

print(f"Mean Squared Error: {mean_squared_error(y_test, y_pred):.2f}")
print(f"R^2 Score: {r2_score(y_test, y_pred):.2f}")
print(f"Model Coefficients: {model.coef_}")
print(f"Model Intercept: {model.intercept_}")

area = float(input("Enter area (sq ft): "))
bedrooms = int(input("Enter number of bedrooms: "))
floors = int(input("Enter number of floors: "))
age = int(input("Enter age of the house: "))
input_data = pd.DataFrame([[area, bedrooms, floors, age]], columns=['area', 'bedrooms', 'floors', 'age'])
input_data_encoded = input_data.reindex(columns=X.columns, fill_value=0)
predicted_price = model.predict(input_data_encoded)

print(f"Predicted price for the house: ₹{predicted_price[0]:,.2f}")
```

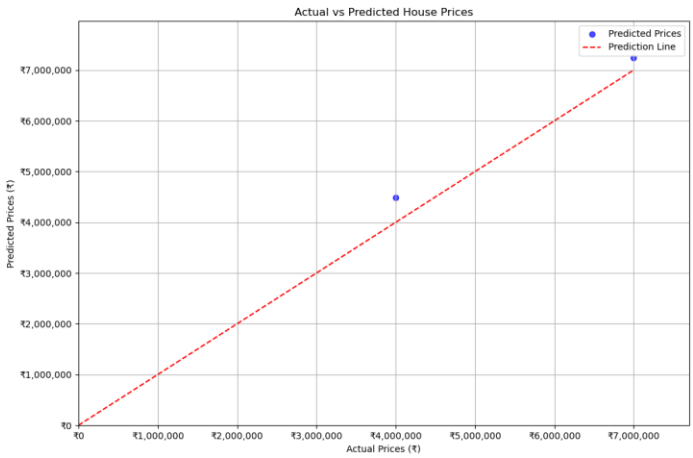
```
plt.figure(figsize=(12, 8)) # Adjusted size
plt.scatter(y_test, y_pred, alpha=0.7, label="Predicted Prices", color='blue', marker='o')
plt.plot([0, max(y_test)], [0, max(y_test)], color='red', linestyle='--', label="Prediction Line")
plt.xlabel("Actual Prices (₹)")
plt.ylabel("Predicted Prices (₹)")
plt.title("Actual vs Predicted House Prices")

# Use safe formatter instead of set_xticklabels/set_yticklabels
formatter = FuncFormatter(lambda x, pos: f'₹{int(x):,}')
plt.gca().xaxis.set_major_formatter(formatter)
plt.gca().yaxis.set_major_formatter(formatter)

plt.legend()
plt.xlim(0, max(y_test) * 1.1)
plt.ylim(0, max(y_pred) * 1.1)
# Grid and display
plt.grid(True)
plt.show()
```

Output:

```
Sample Run:
--------------
$ python3 Multiple_Linear_Regression.py
Mean Squared Error: 147133574968.89
R^2 Score: 0.93
Model Coefficients: [  2518.89168766 198012.87433529 198012.87433529  94458.43828715]
Model Intercept: -818499.8600628739
Enter area (sq ft): 200
Enter number of bedrooms: 3
Enter number of floors: 2
Enter age of the house: 2
Predicted price for the house: ₹864,259.73
```



6. Implementation of Decision tree using sklearn and its parameter tuning

Solution :

A ***Decision Tree*** is a supervised machine learning algorithm used for classification and regression tasks. It builds a tree-like model of decisions based on features of the data, splitting nodes based on conditions that maximize information gain or minimize impurity. Decision Trees are interpretable and useful for understanding the structure of data.

Dataset Overview

The **Iris dataset** is a well-known dataset in machine learning, containing ***150 samples*** from three species of Iris

flowers: *setosa, versicolor, and virginica*. Each sample has four features: *sepal length, sepal width, petal length, and petal width*.

Steps :

1. Importing Required Libraries:

- Libraries from **sklearn** are imported for data handling, model training, evaluation, and visualization.
- matplotlib.pyplot is used for plotting the decision tree.
- load_iris: To load the Iris dataset.
- train_test_split: To split the dataset into training and testing subsets.
- GridSearchCV: To perform hyperparameter tuning through grid search.
- DecisionTreeClassifier: To create a Decision Tree model.
- accuracy_score and classification_report: To evaluate model performance.
- tree: For plotting the tree structure.

2. Loading the Iris Dataset

- The Iris dataset is loaded using load_iris().
- X holds the feature data, and y holds the target labels (species of iris).

3. Splitting the Dataset

- Using train_test_split, 90% of the data is used for training and 10% for testing.
- This ensures the model is evaluated on unseen data.

4. Defining a Parameter Grid for Tuning

- **criterion**: Function to measure split quality - 'gini' or 'entropy'.
- **max_depth**: Max depth of the tree (1 to 4).
- **min_samples_split**: Minimum samples to split an internal node.
- **min_samples_leaf**: Minimum samples required at a leaf node.

5. Initializing the Decision Tree Classifier

- An instance of DecisionTreeClassifier is created with a fixed random state.

6. Performing Grid Search for Parameter Tuning

- GridSearchCV searches through the parameter grid using cross-validation (cv=3).
- It identifies the best combination of parameters for optimal model performance.

7. Getting the Best Model

- best_estimator_ retrieves the best-performing classifier from the grid search.

8. Fitting the Best Classifier on Training Data

- The best classifier is trained using fit() on X_train and y_train.

9. Making Predictions on the Test Set

- Predictions are made using the best model from the grid search.

10. Calculating Accuracy

- accuracy_score is used to evaluate the model's prediction accuracy.

11. Printing Classification Report

- classification_report provides precision, recall, and F1-score for each iris species class.

12. Visualizing the Best Decision Tree

- plot_tree from sklearn.tree is used to visualize the tree structure.
- It includes class names, feature names, and colored nodes for interpretability.
- plt.show() displays the tree plot.

Library Installation:

To install required library files, Open **Command Prompt** or **Terminal** and execute the following commands

```
$ pip install scikit-learn
$ pip install matplotlib
```

Source Code:

File Name: Dtree.py

```
# Importing required libraries
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, classification_report
import matplotlib.pyplot as plt
from sklearn import tree
```

```
# Load the Iris dataset
iris = load_iris()
X = iris.data # Features
y = iris.target # Target labels

# Split the dataset into training and testing sets (90% train, 10% test)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.1, random_state=42) # 10% test size

# Initialize the Decision Tree classifier with some tuning parameters
clf = DecisionTreeClassifier(max_depth=2, min_samples_split=2, min_samples_leaf=1, random_state=42)

# Fit the classifier on the training data
clf.fit(X_train, y_train)

# Make predictions on the test set
y_pred = clf.predict(X_test)

# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)
print(f'Accuracy: {accuracy:.2f}')

# Print classification report
print('Classification Report:')
print(classification_report(y_test, y_pred, target_names=iris.target_names))

# Visualize the Decision Tree
plt.figure(figsize=(10, 8))
tree.plot_tree(clf, filled=True, feature_names=iris.feature_names,class_names=iris.target_names, rounded=True)
plt.title("Decision Tree Visualization")
plt.show()
```

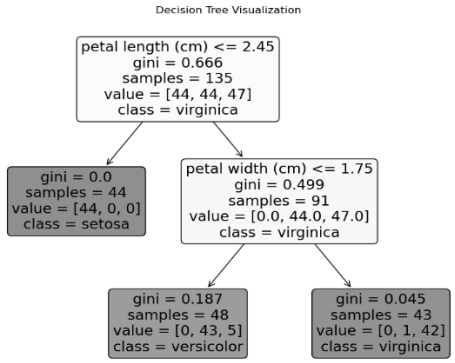
Output:

Sample Run:

```
--------------
$ python3 Dtree.py
Accuracy: 1.00
Classification Report:
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00         6
  versicolor       1.00      1.00      1.00         6
   virginica       1.00      1.00      1.00         3

    accuracy                           1.00        15
   macro avg       1.00      1.00      1.00        15
weighted avg       1.00      1.00      1.00        15
```



7. Implementation of KNN using sklearn

Solution :

K-Nearest Neighbor(KNN) Algorithm:

The K-Nearest Neighbors (KNN) algorithm is a supervised machine learning method employed to tackle classification and regression problems.

K-NN algorithm assumes the similarity between the new case/data and available cases and put the new case into the category that is most similar to the available categories.

K-NN algorithm stores all the available data and classifies a new data point based on the similarity. This means when new data appears then it can be easily classified into a well suite category by using K- NN algorithm.

K-NN algorithm can be used for Regression as well as for Classification but mostly it is used for the Classification problems.

In this program, we are going to use ***iris dataset***.And this dataset Split into training(70%) and test set(30%).

The ***iris dataset*** conatins the following features

- sepal length (cm)
- sepal width (cm)
- petal length (cm)
- petal width (cm)

Library Installation:

To install required library files, Open **Command Prompt** or **Terminal** and execute the following commands

```
$ pip install scikit-learn
```

Source Code:

File Name: Knn.py

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import train_test_split
from sklearn.datasets import load_iris
import random

data_iris = load_iris()
label_target = data_iris.target_names
print()

print("Sample Data from Iris Dataset")
print("*"*30)
for i in range(10):
        rn = random.randint(0,120)
        print(data_iris.data[rn],"===>",label_target[data_iris.target[rn]])
```

```
X = data_iris.data
y = data_iris.target
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.3,random_state=1)

print("The Training dataset length: ",len(X_train))
print("The Testing dataset length: ",len(X_test))
try:
        nn = int(input("Enter number of neighbors :"))
        knn = KNeighborsClassifier(nn)
        knn.fit(X_train, y_train)
        # to display the score
        print("The Score is :",knn.score(X_test, y_test))
        # To get test data from the user
        test_data = input("Enter Test Data :").split(",")
        for i in range(len(test_data)):
                test_data[i] = float(test_data[i])
        print()
        v = knn.predict([test_data])
        print("Predicted output is :",label_target[v])
except:
        print("Please supply valid input.....")
```

Output:

```
Sample Run:
--------------
$ python3 Knn.py
Sample Data from Iris Dataset
*******************************
[6.1 2.8 4.  1.3] ===> versicolor
[4.6 3.1 1.5 0.2] ===> setosa
[5.8 2.6 4.  1.2] ===> versicolor
[6.9 3.2 5.7 2.3] ===> virginica
[6.7 3.1 4.7 1.5] ===> versicolor
[4.7 3.2 1.3 0.2] ===> setosa
[5.  3.  1.6 0.2] ===> setosa
[6.4 2.9 4.3 1.3] ===> versicolor
[5.5 2.4 3.7 1. ] ===> versicolor
[5.8 2.6 4.  1.2] ===> versicolor
The Training dataset length:  105
The Testing dataset length:  45
Enter number of neighbors :9
The Score is : 0.9777777777777777
Enter Test Data :5.3,2.2,3.2,1.5

Predicted output is : ['versicolor']
```

8. Implementation of Logistic Regression using sklearn

Solution :

Logistic regression:

Logistic regression is one of the most popular Machine Learning algorithms, which comes under the Supervised Learning technique. It is used for predicting the categorical dependent variable using a given set of independent variables. Logistic regression predicts the output of a categorical dependent variable. Therefore the outcome must be a categorical or discrete value. It can be either Yes or No, 0 or 1, true or False, etc. but instead of giving the exact value as 0 and 1, it gives the probabilistic values which lie between 0 and 1.

In this implementation, we use "Study Hours" as the independent variable and "Exam Results (0 or 1)" as the dependent variable.

Steps :

Step 1. Importing Required Libraries

- **pandas**: For loading and processing the dataset.
- **numpy**: For numerical computations, such as generating a range of values for plotting.
- **matplotlib.pyplot**: For creating the graph and visualizing the results.
- **sklearn.model_selection**: For splitting the dataset into training and testing subsets.
- **sklearn.linear_model**: For implementing the logistic regression model.
- **sklearn.metrics**: For evaluating the model's performance using metrics like accuracy, confusion matrix, and classification report.

Step 2. Reading the Dataset

- pd.read_csv() reads the CSV file into a Pandas DataFrame.
- The file should contain columns for "Study Hours" and "Exam Result".

Step 3. Defining Features (X) and Target (y)

- **X** stores the independent variable (Study Hours).
- **y** stores the dependent variable (Exam Result).

Step 4. Splitting the Dataset

- train_test_split() is used to split the data into training (80%) and testing (20%) sets.
- random_state=42 ensures the split is reproducible.

Step 5. Training the Logistic Regression Model

- LogisticRegression() initializes the logistic regression model.
- model.fit(X_train, y_train) trains the model on the training data.

Step 6. Making Predictions

- model.predict(X_test) uses the trained model to predict the exam results for the test set.

Step 7. Evaluating the Model

- **Accuracy**: The percentage of correct predictions, calculated by accuracy_score().
- **Confusion Matrix**: Shows TP, TN, FP, FN for insight into performance.
- **Classification Report**: Provides precision, recall, F1-score, and support for each class.

Step 8. Plotting the Data Points and Decision Boundary

- **Scatter Plot**: Displays training data in blue and test data in green (x-shaped).
- **Decision Boundary**: Red line showing model's predicted probabilities for class 1 across study hours.

Library Installation:

To install required libraries, Open **Command Prompt** or **Terminal** and execute the following commands

```
$ pip install scikit-learn
$ pip install numpy
$ pip install pandas
$ pip install matplotlib
```

CSV file : "study_hours.csv"

```
Study Hours,Exam Result
1,0
2,0
3,0
4,0
5,1
6,1
7,1
8,1
9,1
10,1
```

9. Implementation of K-Means Clustering

Solution :

K-Means Clustering:

K-Means is a popular unsupervised machine learning algorithm used for partitioning a dataset into K clusters based on the similarity of their features. It works by minimizing the variance within each cluster and is widely used in applications such as market segmentation, image compression, and pattern recognition.

K means clustering, assigns data points to one of the K clusters depending on their distance from the center of the

clusters. It starts by randomly assigning the clusters centroid in the space. Then each data point assign to one of the cluster based on its distance from centroid of the cluster. After assigning each point to one of the cluster, new cluster centroids are assigned. This process runs iteratively until it finds good cluster. In the analysis we assume that number of cluster is given in advanced and we have to put points in one of the group.

ALGORITHM:

1. **Initialization:**
   - Choose the number of clusters (k) to divide the data into.
   - Initialize the centroids (cluster centers) randomly or using specific techniques.
2. **Assignment:**
   - Assign each data point to the nearest centroid based on the distance (e.g., Euclidean distance).
   - Each data point is labeled as belonging to one of the k clusters.
3. **Update:**
   - Recalculate the centroids of the clusters by averaging the coordinates of all data points assigned to each cluster.
4. **Repeat:**
   - Repeat the Assignment and Update steps iteratively until the centroids stabilize (i.e., no significant changes in their positions) or a predefined number of iterations is reached.

Steps for Program:

1. **Import Necessary Libraries:**
   - numpy: For numerical computations.
   - matplotlib.pyplot: For visualizing the data points and clustering results.
   - KMeans from sklearn.cluster: To implement the K-Means algorithm.
   - make_blobs from sklearn.datasets: To generate synthetic data for clustering.
2. **Generate Synthetic Data:**
   - The make_blobs function generates 300 data points with 4 clusters. The points are distributed with a standard deviation of 0.60 around the cluster centers.
   - The dataset consists of:

Features (X): A 2D array representing the data points' coordinates.

3. **Visualize Raw Data:**
   - A scatter plot is created to visualize the data points before clustering.
4. **Apply K-Means Algorithm:**
   - The K-Means model is initialized with n_clusters=4 to specify 4 clusters.
   - The model is fit to the data using the fit method.
5. **Retrieve Cluster Information:**
   - **Cluster Centers:** The coordinates of the centroids for each cluster are stored in kmeans.cluster_centers_.
   - **Cluster Labels:** Each data point is assigned a cluster label using kmeans.labels_.
6. **Visualize Clustered Data:**
   - A scatter plot shows the data points grouped into clusters, with colors representing different clusters.
   - Cluster centroids are displayed as red points to highlight their locations.

Library Installation:

To install required libraries, Open **Command Prompt** or **Terminal** and execute the following commands

```
$ pip install scikit-learn
$ pip install numpy
$ pip install matplotlib
```

Source Code:

File Name: kmeans.py

Copy

```
#Necessary Libraries
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs

#Generate Data
```

```
X, _ = make_blobs(n_samples = 300, centers = 4 , cluster_std = 0.60,random_state = 0)

#Plot the data points
plt.scatter(X[:,0],X[:,1])
plt.title("Data Points")
plt.show()

#KMeans clustering
kmeans = KMeans(n_clusters = 4)
kmeans.fit(X)

#Getting the cluster centers and labels
centers = kmeans.cluster_centers_ #returns the coordinates of the

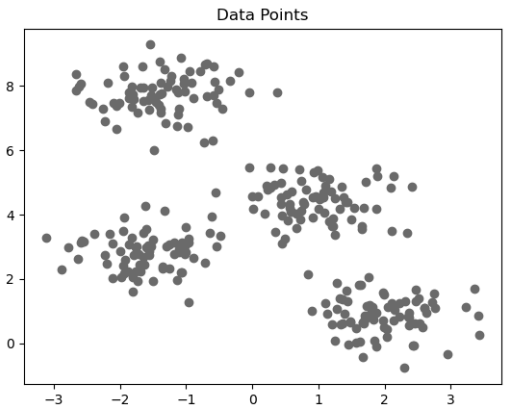
#cluster center found by the algorithm
labels = kmeans.labels_ #Returns the labels for each data point , indicating which cluster it has

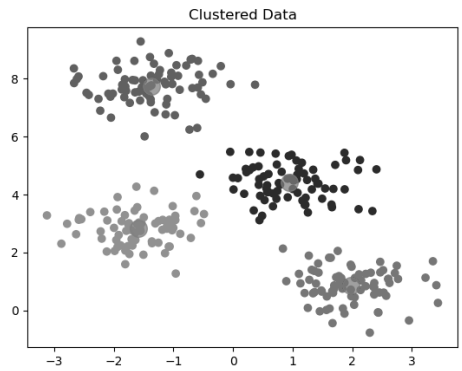
#been assigned to.
#Plot the clustered data
plt.scatter(X[:,0],X[:,1],c = labels)
plt.scatter(centers[:,0],centers[:,1],alpha = 0.75,s=200,color = 'red')
plt.title("Clustered Data")
plt.show()
```

Output:

Sample Run:

$ python3 kmeans.py





10. Performance analysis of Classification Algorithms on a specific dataset (Mini Project)

Mini Project : Performance Analysis of Classification Algorithms on the Iris Dataset.

Title: Performance Analysis of Classification Algorithms on the Iris Dataset
Objective:
To evaluate and compare the performance of various classification algorithms such as:

- Logistic Regression
- Decision Tree
- K-Nearest Neighbors (KNN)
- Support Vector Machine (SVM)
- Random Forest

using performance metrics like accuracy, confusion matrix, precision, recall, and F1-score.
Dataset:

- **Dataset Name:** Iris Dataset
- **Source:** sklearn.datasets.load_iris() or UCI Machine Learning Repository
- **Features:** Sepal Length, Sepal Width, Petal Length, Petal Width
- **Target:** Iris species (setosa, versicolor, virginica)

Tools & Libraries:

```
import numpy as np
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score
```

Step-by-Step Implementation:
1. Load and Prepare Data

```
iris = load_iris()
X = iris.data
y = iris.target

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Feature scaling
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

2. Initialize Models

```
models = {
    "Logistic Regression": LogisticRegression(),
    "Decision Tree": DecisionTreeClassifier(),
    "K-Nearest Neighbors": KNeighborsClassifier(),
    "Support Vector Machine": SVC(),
    "Random Forest": RandomForestClassifier()
}
```

3. Train, Predict, and Evaluate

```
for name, model in models.items():
    print(f"\nModel: {name}")
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)

    print("Accuracy:", accuracy_score(y_test, y_pred))
```

```
    print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
    print("Classification Report:\n", classification_report(y_test, y_pred))
```

Sample Output (for SVM):

```
Model: Support Vector Machine
Accuracy: 1.0
Confusion Matrix:
[[16  0  0]
 [ 0 14  0]
 [ 0  0 15]]
Classification Report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        16
           1       1.00      1.00      1.00        14
           2       1.00      1.00      1.00        15

    accuracy                           1.00        45
   macro avg       1.00      1.00      1.00        45
weighted avg       1.00      1.00      1.00        45
```

Metrics Used for Comparison:

- Accuracy
- Confusion Matrix
- Precision
- Recall
- F1-score

Conclusion:

Based on the evaluation metrics, all models may perform well on this simple dataset, but on real-world data, metrics will vary. Random Forest and SVM generally provide better performance in complex datasets.

Optional Extensions:

- Use different datasets: Breast Cancer, Wine, Titanic (Kaggle)
- Add cross-validation
- Use AUC-ROC curves for binary classification
- Plot confusion matrix using seaborn heatmap

Complete Source Code:

File Name: mini_project.py

```
import numpy as np
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score

# Load dataset
iris = load_iris()
X = iris.data
y = iris.target

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Feature scaling
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

```
# Define models
models = {
    "Logistic Regression": LogisticRegression(),
    "Decision Tree": DecisionTreeClassifier(),
    "K-Nearest Neighbors": KNeighborsClassifier(),
    "Support Vector Machine": SVC(),
    "Random Forest": RandomForestClassifier()
}

# Train, Predict, and Evaluate
for name, model in models.items():
    print(f"\nModel: {name}")
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)

    print("Accuracy:", accuracy_score(y_test, y_pred))
    print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
    print("Classification Report:\n", classification_report(y_test, y_pred))
```

Output:

Sample Run:

```
$ python3 mini_project.py
Model: Logistic Regression
Accuracy: 1.0
Confusion Matrix:
 [[19  0  0]
 [ 0 13  0]
 [ 0  0 13]]
Classification Report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        19
           1       1.00      1.00      1.00        13
           2       1.00      1.00      1.00        13

    accuracy                           1.00        45
   macro avg       1.00      1.00      1.00        45
weighted avg       1.00      1.00      1.00        45


Model: Decision Tree
Accuracy: 1.0
Confusion Matrix:
 [[19  0  0]
 [ 0 13  0]
 [ 0  0 13]]
Classification Report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        19
           1       1.00      1.00      1.00        13
           2       1.00      1.00      1.00        13

    accuracy                           1.00        45
   macro avg       1.00      1.00      1.00        45
weighted avg       1.00      1.00      1.00        45
```

Model: K-Nearest Neighbors
Accuracy: 1.0
Confusion Matrix:
[[19 0 0]
[0 13 0]
[0 0 13]]
Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	19
1	1.00	1.00	1.00	13
2	1.00	1.00	1.00	13
accuracy			1.00	45
macro avg	1.00	1.00	1.00	45
weighted avg	1.00	1.00	1.00	45

Model: Support Vector Machine
Accuracy: 1.0
Confusion Matrix:
[[19 0 0]
[0 13 0]
[0 0 13]]
Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	19
1	1.00	1.00	1.00	13
2	1.00	1.00	1.00	13
accuracy			1.00	45
macro avg	1.00	1.00	1.00	45
weighted avg	1.00	1.00	1.00	45

Model: Random Forest
Accuracy: 1.0
Confusion Matrix:
[[19 0 0]
[0 13 0]
[0 0 13]]
Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	19
1	1.00	1.00	1.00	13
2	1.00	1.00	1.00	13
accuracy			1.00	45
macro avg	1.00	1.00	1.00	45
weighted avg	1.00	1.00	1.00	45